

VSEPR-SIM

Engineering Scope Reference

Output Stack • .dynx Pipeline • .X Bundle • v5.1.x

W0-OUTPUT-P2 • W0-72A • W0-72B • W0-VSIM-CTL • W0-67-B

Document scope. This reference covers the implemented output and viewer pipeline as it exists in the v5.0.0-main / v5.1.x codebase: the demo artifact stack (W0-OUTPUT-P2), the .X suite bundle format (W0-72A), the .dynx rich-frame emitter pipeline (W0-72B), the modernised VSIM control pipeline (W0-VSIM-CTL), and the viewer file-type data contract (W0-67-B). All sections describe code that is *built, tested, and passing* — not planned features.

Contents

1	WO-OUTPUT-P2 Output System Expansion — Phase 2: Demo Stack	3
2	WO-72A .X Bundle Format	5
3	WO-72B .dynx Pipeline Emitter	7
4	WO-67-B Viewer File-Type Data Contract	9
5	WO-VSIM-CTL Modernised VSIM Control Pipeline	11
6	VsimDocument Schema Overview	12
7	Cross-Reference: Output Formats and Viewer Pipeline	14

WO-OUTPUT-P2 Output System Expansion — Phase 2: Demo Stack

COMPLETE

SCHEMA

IO LAYER

Purpose

Produce condensed, self-contained molecular demonstration artifacts from batch and single simulation runs. A “demo artifact” is a lightweight snapshot of a run — not a replay of the full trajectory — suitable for embedding in reports, CI pipelines, and interactive review. The stack adds three new layers on top of the existing .dynx writer: a schema section, a frame sampler, and a bundle writer.

Schema: [export.demo] — ExportDemoSection

Declared in include/vsim/vsim_document.hpp. All fields have safe defaults; the section is off by default (enabled = false).

```
struct ExportDemoSection {
    bool        enabled          = false;    // master switch
    int         demo_frames      = 10;       // max frames to include
    std::string strategy         = "uniform";
    // "first" | "last" | "uniform" | "event_gated"
    bool        write_demo_dynx  = true;     // emit <case>.demo.dynx
    bool        write_demo_bundle = true;    // emit <case>.demo.X
    bool        include_source   = true;     // embed .vsim in bundle
    bool        include_manifest = true;     // embed demo_manifest.json
    std::string bundle_name;           // override name (auto if empty)
};
```

VSIM config block:

```
[export.demo]
enabled          = true
demo_frames      = 12
strategy         = "uniform"      # first | last | uniform | event_gated
write_demo_dynx  = true
write_demo_bundle = true
include_source   = true
include_manifest = true
```

DemoFrameSampler — include/vsim/io/demo_frame_sampler.hpp

Selects frames from a DynxReader archive according to ExportDemoSection::strategy:

Strategy	Behaviour
first	Frames $0 \dots \min(\text{demo_frames} - 1, N - 1)$
last	Last <i>demo_frames</i> frames from the archive
uniform	Evenly spaced indices across the full archive (default)
event_gated	Only frames containing at least one EVENT packet

The sampler operates on already-written .dynx archives; it does not buffer frames in memory.

DemoBundleWriter — include/vsim/io/demo_bundle_writer.hpp

Consumes sampled frames from DemoFrameSampler and emits two artifacts:

1. <case>.demo.dynx — a condensed .dynx archive containing only the sampled frames, in standard Dynx v1 format. Readable by any DynxReader.
2. <case>.demo.X — a self-contained .X bundle embedding the .demo.dynx file, the originating .vsim script, and a demo_manifest.json provenance record.

Batch aggregator hook

`run_demo_artifacts()` is called by the batch post-run hook after each run completes. It resolves the run's `.dynx` path, instantiates a `DemoFrameSampler` and `DemoBundleWriter`, and writes the demo artifacts into the run output directory. The call is gated on `ExportDemoSection::enabled` and is a no-op when the section is absent.

Output files

```
<run_dir> / <case> .demo.dynx(condensedDynxv1archive) <run_dir> / <case> .demo.X(self -
containedbundle) <run_dir> /demo_manifest.json(provenance : source, strategy, frameindices)
```

Test coverage

Group	Name	Assertions	Coverage
73	ExportDemoSectionGroup73	25	Schema defaults, parser round-trip, all strategy strings
74	DemoFrameSamplerGroup74	21	Sampler strategies, reduced archive generation, edge cases
75	DemoBundleWriterGroup75	19	Bundle round-trip, manifest embedding, source inclusion

Build gate: All three groups pass. Full `ninja` build exits 0 with zero regressions.

WO-72A .X Bundle Format

COMPLETE

FORMAT

Purpose

The .X format is a bundled suite-execution container. It packs one or more .vsim scripts (or inline script text), optional assets, dependency ordering, and expected output declarations into a single text file that the vsepr CLI can execute as an ordered unit. It is also a reproducibility artifact: it carries the run seed, version tag, and a SHA-1 hash of each included script so that replaying the bundle produces identical outputs given the same binary.

Role separation

Format	Role
.vsim	Human-authored source script
.xyz / .xyzFull	Scientific state / replay truth
.dynx	Live visual / session archive (post-compiled)
.X	Bundled suite-execution container

Data model — include/vsim/bundle/x_bundle.hpp

```
// Slot kinds
enum class XBundleSlotKind : uint8_t {
    Script, InlineScript, PyKernel, ValidationCheck, Benchmark
};

struct XBundleSlot {
    std::string id;           // unique slot id within bundle
    std::string label;       // human label, e.g. "ANN-01 identity gate"
    XBundleSlotKind kind;
    std::string script_path;  // for Script / PyKernel kinds
    std::string inline_text;  // for InlineScript kind
    std::vector<XBundleOverride> overrides;
    std::vector<std::string> depends_on; // ordered dependency ids
    std::vector<XBundleOutput> expected_outputs;
    double max_wall_s = 0.0; // 0 = no limit
    bool allow_failure = false;
    std::string script_hash_sha1; // verified before execution
};

struct XBundleManifest {
    std::string bundle_id; // script-defined; no hardcoded defaults
    std::string version;
    std::string purpose;
    std::string author;
    std::string created_utc;
    uint64_t run_seed = 0;
    std::string vsim_version;
    std::string git_ref;
    std::vector<XBundleSlot> slots;
    std::string summary_md_path;
    std::string events_jsonl_path;
    std::string bench_tsv_path;
    bool abort_on_first_failure = false;
    double global_wall_limit_s = 0.0;
};
```

Slot outcomes

Outcome	Meaning
Pending	Not yet started
Running	Currently executing
Passed	Completed; all expected outputs present and valid
Failed	Slot or output assertion failed
Skipped	A dependency failed; slot was bypassed

Typical use cases

- ✓ Annihilation test ladders (ANN-01..10)
- ✓ Formation sweeps with multiple seeds
- ✓ Benchmark suites run by CI
- ✓ Multi-step validation chains
- ✓ Demo bundle embedding (WO-OUTPUT-P2)
- ✓ Post-compiled .dynx + .vsim pairs

Annihilation test ladder factory

The header provides `make_annihilation_test_bundle(seed, script_dir)` which constructs the canonical ANN-01 through ANN-10 slot sequence with correct dependency ordering. Slot ANN-04 depends on ANN-02 *and* ANN-03 (both gates must pass before product records are created).

Test coverage

Group 70 (XBundledSmokeGroup70) — 10 assertions covering manifest construction, slot lookup, dependency ordering, and outcome classification. **144/144 tests pass. Zero regressions.**

WO-72B .dynx Pipeline Emitter

COMPLETE

IO LAYER

ARCHIVE + LIVE

Purpose

The .dynx format is a post-compiled dynamic session archive. WO-72B adds the *emitter pipeline*: a post-step hook that consumes rich simulation state each FIRE step and simultaneously writes a persistent .dynx archive *and* pushes the latest frame into a thread-safe live cache polled by the viewer.

Dynx v1 wire format

```
#dynx v1
#source <path>
#source_hash <sha256-hex | "none">
#kernel_version <string>
#frame_count <N>
#frame_interval <dt_fs>
#particle_count <N>
#timestamp <ISO-8601>
FRAME <index> <time_fs>
<N> <symbol> <x> <y> <z> [<vx> <vy> <vz>] [<energy>]
FORCE      <idx> <fx> <fy> <fz>          (optional)
BOND_FORCE <i> <j> <bfx> <bfi> <bfi>      (optional)
FIELD      <label> <fx> <fy> <fz>        (optional)
EVENT      <kind> <event_id> <source> <value> (optional)
RENDER     <idx> <r> <g> <b> <tag> <visible> (optional)
CAMERA     <label> <x> <y> <z> <pitch> <yaw> <zoom> (optional)
END_FRAME
...
#END_DYNX
```

Optional lines (FORCE, BOND_FORCE, FIELD, EVENT, RENDER, CAMERA) are forward-compatible: unknown prefixes are silently skipped by v1 readers. Velocities and per-particle energy are also optional.

Rich-frame data structs — include/vsim/io/dynx_writer.hpp

```
struct DynxForceState { int idx; std::array<double,3> force; };
struct DynxBondForce { int i, j; std::array<double,3> force; };
struct DynxFieldVector { std::string label; std::array<double,3> vec; };
struct DynxEventPacket {
    std::string kind;    // "reaction" | "checkpoint" | "phase_transition"
    uint64_t event_id = 0;
    std::string source;  // source_formula from KernelEvent
    double value = 0.0;
};
struct DynxRenderMeta {
    int particle_idx;
    uint8_t r, g, b;    // RGB colour override
    std::string tag;
    bool visible = true;
};
struct DynxCameraState {
    std::string label;  // "default" | "close_up" | ...
    // x, y, z, pitch, yaw, zoom
};
struct DynxRichFrame {
    DynxFrame base;
    std::vector<DynxForceState> forces;
    std::vector<DynxBondForce> bond_forces;
    std::vector<DynxFieldVector> field_vectors;
    std::vector<DynxEventPacket> events;
```

```
std::vector<DynxRenderMeta> render_meta;
std::vector<DynxCameraState> cameras;
};
```

Emitter architecture

```

FIRE step  → DynxEmitter::emit_step() → DynxWriter::write_rich_frame()
           → DynxLiveCache::push()

```

DynxWriter Streams rich frames to a .dynx file. Backward-compatible with v1 baseline readers.

DynxEmitter Post-step hook. Calls `KernelEventLog::instance().filter_by_frame(fid, fid)` to harvest matching events automatically, then calls `write_rich_frame()`. Pushes a copy of the frame to `DynxLiveCache`.

DynxLiveCache Mutex-guarded single-slot cache. `push()` replaces any existing slot. `poll()` returns the frame *and clears the slot*. The viewer calls `poll()` each render tick; no frame is stale.

Live cache without archive. `emit_step()` without a prior `open()` call still pushes to `DynxLiveCache` and returns `true`. No archive file is required for the live viewer path.

KernelEvent harvesting

`DynxEmitter::emit_step()` calls:

```
KernelEventLog::instance().filter_by_frame(frame_id, frame_id)
```

Events matching the current frame are automatically appended as `EVENT` packets in the rich frame, in addition to any caller-supplied events in `DynxEmitContext`.

Test coverage

Group 71 (`DynxEmitterGroup71`) — 15 test cases, 29 assertions (71-A through 71-O): live cache push/poll/clear, rich-frame write/read round-trip, event harvesting, archive-open gating, and full emitter pipeline. **145/145 tests pass. Zero regressions.**

WO-67-B Viewer File-Type Data Contract

COMPLETE

CONTRACT LAYER

Purpose

The viewer is a *dumb renderer*: it reads decoded state from a canonical `ViewFrame` and renders it. It never reads simulation internals directly and never mutates physical state. WO-67-B defines the authoritative contract between each supported file type and the `ViewFrame`, specifying what each loader *must* and *may* contribute.

File-type contract table

Format	Role	ViewFrame contribution
.xyz	Static particle positions	1 frame, particle labels, positions
.xyza	Enriched particle state	Positions + charge/velocity/force/energy
.xyzf	Trajectory	N frames, frame stepping
.xyzFull	Rich replay	Trajectory + metadata + event refs
.xyzc	Checkpoint geometry	1+ frames, checkpoint observables
.dynx	Live visual / session archive	N frames + render metadata
.z	Identity matrix overlay	Identities only (linked by id)
.ee	Electron / subatomic overlay	Identities only (linked by id)
.X	Bundled run container	Dispatches to loaders above

Contract rules

- ✓ .xyz MUST be viewable alone
- ✓ .xyzf MUST animate ($N \geq 1$ frames)
- ✓ .xyzFull MUST replay with metadata (observables, comment-line tags)
- ✓ .z / .ee MUST overlay only when linked to particle IDs — never evolve state
- ✓ .X MUST route to the appropriate loader for each contained entry
- ✓ ALL types: viewer MUST NOT mutate physical state (`must_not_mutate_state = true`)

Viewer data model — `include/vsim/view/viewer_types.hpp`

The render pipeline uses three leaf structs:

```
struct ViewParticle {
    int64_t id, identity_id;    // 0 = no identity sidecar
    std::string type;          // element symbol or bead tag
    double x, y, z;            // Angstrom
    double vx, vy, vz;         // Angstrom/fs
    double radius, charge, energy;
    uint32_t type_code;        // Atom=0 | Bead=1 | CoarseBead=2 | Virtual=3
    uint32_t flags;            // Selected | Hidden | Highlighted | Defect | ...
};

struct ViewBond {
    int64_t a, b;
    double order, strength;
    double fx, fy, fz;        // bond-force vector (eV/Angstrom)
    double lifetime;          // -1 = permanent
};
```

```
    uint32_t flags;
};

struct ViewIdentity {                // .z / .ee sidecar overlay
    int64_t identity_id;
    double  Z2[4];                  // 2x2 Z-matrix (row-major)
    double  Y, loss;
    uint32_t flags;
};

struct ViewFrame {
    int64_t frame_index;
    double  time, dt;               // fs
    std::vector<ViewParticle> particles;
    std::vector<ViewBond> bonds;
    std::vector<ViewIdentity> identities;
    std::vector<std::string> warnings;
    std::unordered_map<std::string, double> observables;
};
```

ViewSession — deterministic replay

ViewSession holds all frames for a loaded file plus loader metadata. ViewSession::deterministic_hash() computes a stable identity hash from particle count, frame count, and the first-frame coordinate centroid, enabling replay validation without byte-for-byte file comparison.

WO-VSIM-CTL Modernised VSIM Control Pipeline

COMPLETE

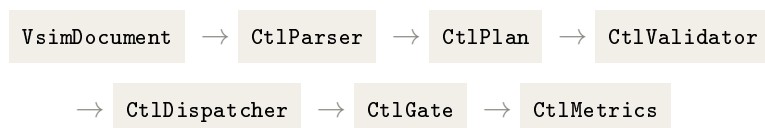
RUNTIME

Purpose

The CTL layer is the VSIM runtime control pipeline: it parses, validates, dispatches, and gates .vsim scripts through the simulation kernel. WO-VSIM-CTL modernised this layer with a formal type system, a validator, a metrics collector, and a gate / dispatch model that is script-driven rather than hard-coded.

Headers — `include/vsim/ctl/`

File	Role
ctl_types.hpp	CtlKind, CtlParam, CtlDirective — fundamental types
ctl_plan.hpp	CtlPlan — ordered list of directives from a parsed script
ctl_parser.hpp	CtlParser::parse() — converts VsimDocument to CtlPlan
ctl_validator.hpp	CtlValidator::validate() — structural correctness checks
ctl_dispatcher.hpp	CtlDispatcher::dispatch() — routes directives to kernel calls
ctl_gate.hpp	CtlGate — pass/fail gate on simulation state
ctl_metrics.hpp	CtlMetrics — per-directive wall time and step counts

Pipeline flow

The validator runs *before* dispatch; invalid plans never reach the kernel. Gates are evaluated *after* each directive completes; a failing gate aborts the plan and records the failure reason in CtlMetrics.

VsimDocument section interaction

The CTL parser consumes the following VsimDocument sections in order:

- [project] — identity, seed
- [simulation] — molecule list, FIRE config
- [batch] / [batch.design] — sweep axes, cases
- [while] / [until] / [loop] — control guards
- [export] / [export.visual] — output targets
- [export.demo] — demo artifact config
- [visual] — display mode
- [variance] / [N_evolution] — probe declarations
- [select] — Uitt2 weighted scoring

VsimDocument Schema Overview

REFERENCE

V5.1.X

Purpose

include/vsim/vsim_document.hpp is the single schema source of truth. Every .vsim section maps to a plain aggregate struct with no virtual dispatch or hidden state. The table below lists every implemented section.

Section	Struct	Notes
[project]	ProjectSection	name, seed, world_seed, script_type
[simulation]	SimulationSection	FIRE config, molecule list, Ewald
[[simulation.molecule]]	MoleculeEntry	formula, count, lattice, region, drift
[cell]	CellSection	orthorhombic box; lx/ly/lz in Å
[boundary]	BoundarySection	per-axis: open/periodic/reflective
[pbc]	PBCSection	minimum_image, wrap_positions, images
[export]	ExportSection	xyz, xyzFull, JSON, TSV, audit, etc.
[export.visual]	ExportVisualSection	SVG, PNG, GIF, HTML, WebGL, reports
[export.demo]	ExportDemoSection	WO-OUTPUT-P2 demo artifact stack
[visual]	VisualSection	terminal/GL/web output_type
[visual.external]	VisualExternalSection	render_targets, export_format
[visual.workspace]	VisualWorkspaceSection	host layout, live subscriptions
[open]	OpenSection	post-run viewer launch
[room]	RoomSection	room heat-field preset + steps
[variance]	VarianceSection	statistical spread probes
[N_evolution]	NEvolutionSection	dN/dt population tracking
[while]	WhileSection	conditional continuation guards
[until]	UntilSection	run-until convergence guards
[loop]	SmartLoopSection	deterministic / stochastic sweep
[select]	SelectSection	Uitt2 weighted material scoring
[batch]	BatchSection	parameter sweep job sets
[batch.design]	BatchDesignSection	factorial/grid/latin_hypercube
[[batch.axis]]	BatchAxisEntry	static/stochastic/formation axes
[[batch.case]]	BatchCaseEntry	named override cases
[study]	StudySection	top-level study metadata
[seed]	SeedSection	foundation/defect/thermal/placement
[formation.library]	FormationLibraryEntry	hold/ramp/quench/anneal/relax stages
[defaults.run]	GoldenDefaultsSection	golden test run defaults
[test.name]	GoldenTestEntry	golden test registry entry
[suite]	SuiteSection	group selection, limits, smoke
[report]	GoldenReportSection	report content flags

Design rule

`VsimDocument` stores *what the script says*. The runner decides what to do with it. No section struct contains runtime state, thread handles, file handles, or virtual methods. Every field has a documented default and is individually inspectable without executing any simulation code.

Cross-Reference: Output Formats and Viewer Pipeline

Format roles

Format	Produced by	Consumed by	Notes
.vsim	Engineer (authored)	CtlParser, VsimParser	Source script; never generated
.xyz	ExportSection	XyzViewLoader, external tools	Static truth; standard tools read it
.xyzFull	ExportSection	XyzViewLoader	Rich replay with metadata
.dynx	DynxWriter	DynxReader, DynxLiveCache, viewer	Post-compiled; carries render meta-data
.demo.dynx	DemoBundleWriter	DynxReader, viewer, CI	Condensed demo subset of a full archive
.X	XBundleWriter	vsepr CLI, CI runner	Suite container
.demo.X	DemoBundleWriter	vsepr CLI	Self-contained demo bundle
.z	Kernel identity layer	viewer (overlay only)	Z-matrix sidecar; never parsed alone
.ee	Subatomic kernel layer	viewer (overlay only)	Electron state sidecar

Work-order completion status

WO	Status	Build gate	Key deliverable
WO-OUTPUT-P2	COMPLETE	65 tests pass (Gr. 73–75)	ExportDemoSection, DemoFrameSampler, DemoBundleWriter
WO-72A	COMPLETE	144/144 pass (Gr. 70)	XBundleManifest, XBundleSlot, slot runner
WO-72B	COMPLETE	145/145 pass (Gr. 71)	DynxEmitter, DynxLiveCache, rich-frame write/read
WO-67-B	COMPLETE	contract enforced	ViewFileContract, ViewFrame, ViewSession
WO-VSIM-CTL	COMPLETE	full build exit 0	CtlPlan, CtlParser, CtlDispatcher, CtlGate

One-paragraph framing. The VSEPR-SIM output and viewer pipeline consists of five completed work orders. WO-OUTPUT-P2 adds a three-layer demo artifact stack (`ExportDemoSection` schema, `DemoFrameSampler`, `DemoBundleWriter`) that produces condensed `.demo.dynx` archives and self-contained `.demo.X` bundles from any batch run. WO-72A defines the `.X` suite-execution container: a text archive of ordered `.vsim` slots with dependency ordering, override parameters, and SHA-1 reproducibility hashes. WO-72B adds the `DynxEmitter` post-step hook which simultaneously streams rich frames (forces, bond-forces, field vectors, event packets, render metadata, camera states) to a persistent `.dynx` archive *and* pushes each frame to a mutex-guarded live cache polled by the viewer at render cadence. WO-67-B establishes the viewer file-type data contract: a formal specification of what each loader must and may contribute to the canonical `ViewFrame` that the renderer consumes. WO-VSIM-CTL modernises the runtime control pipeline with a typed plan model, validator, dispatcher, and gate layer that converts any `VsimDocument` into an executable sequence of kernel calls without hard-coded dispatch logic.